Multi-objective cooperative co-evolution of micro for RTS games

Navin K. Adhikari, Sushil J. Louis, and Siming Liu
Department of Computer Science and Engineering
University of Nevada, Reno, USA
navinadhikari@nevada.unr.edu, sushil@cse.unr.edu, and simingl@cse.unr.edu

***Abstract*—We investigate a multi-objective, cooperative, co-evolutionary algorithm to evolve control tactics for groups composed from multiple types of units in real-time strategy games. Such tactical control or micromanagement of units is called micro. Building on prior work, we cooperatively co-evolve micro for a ranged unit using a parameterized control algorithm along with micro for a melee unit using a pure potential fields approach and show that we can simultaneously co-evolve micro for melee and ranged units. These cooperatively co-evolved control algorithms for melee and ranged units evolve to work well together to defeat the default Starcraft II AI, even when outnumbered. We are also able to generate manually co-evolved AI that is significantly better than the default Starcraft II AI and defeat it to generate human competitive micro for controlling multiple types of units. Furthermore, using a multi-objective fitness function leads to a pareto front of near-optimal micro behaviors that range from fleeing while sustaining minimal damage to fighting and maximizing damage to opponents. Such a pareto front naturally provides a user or AI player a variety of micro behaviors suitable for the different types of situations encountered in real-time strategy games. We believe these results indicate the potential of our method for generating effective micro for multiple types of units in real-time strategy games with application in multi-agent control, robotics, and other heterogeneous system control problems.**

***Index Terms*—Cooperative co-evolution, RTS games, NSGA II, Micro.**

I. Introduction

Real-Time Strategy (RTS) games provide a complex dynamic environment with many real-world properties that need to be addressed in order to build robust intelligence. Expert level gameplay requires multi-tasking in an uncertain environment under resource and real-time constraints [1]. In order to defeat an opponent, an RTS player requires real-time planning on several levels against a deceptive, dynamic, and adaptive opponent. RTS players usually divide such planning into two broad categories: macro-management and micro-management. Macro-management, or just macro for short, encompasses long-term plans that include tasks like collecting resources, constant unit production, technology upgrades, and scouting. In contrast, micro-management, or just micro for short, encompasses short-term control and reactive tasks like navigating units, proper target selection, and group or unit control based on properties of the group and individual units.

Our research is motivated by the goal of building a Computational Intelligence (CI) agent that is capable of performing equal to a human expert in this complex dynamic environment. Although both good long-term planning and good short-term control play important roles in a game, our current work focuses on micro. Good micro can win skirmishes even when a player has fewer units or when units are unfavourably positioned.

Earlier work had shown promise in evolving micro for a group of ranged units using a parameterized control algorithm for target selection, kiting and fleeing [2]. But this method had been specifically designed to evolve micro for ranged units. Another approach using a Pure Potential Fields (PPF) representation was able to evolve micro for a group of melee and ranged units [3]. This approach also used a two objective fitness function that sought to maximize damage done to enemy units while minimizing damage to friendlies. This Evolutionary Multi-Objective Optimization (EMOO) algorithm works well but required a chromosome length that grew as the square of the number of types of units. In this paper, we investigate a cooperative co-evolutionary approach whose chromosome length only grows linearly in the number of different unit types. In our Multi-Objective Cooperative Co-Evolutionary Algorithm (MOCCEA), rather than evolving one single population that represents the global solution, several sub-populations representing specific parts of the global solution, in our case specific unit types, cooperatively co-evolve good multi-unit micro.

Our approach has two significant advantages over traditional EMOOs for evolving micro for a group having multiple types of units. First, we don't need one general representation of micro for different types of units to evolve. RTS game units have a diverse range of attributes and abilities which often makes it non-trivial to come up with one general representation. To handle this, MOCCEAs can evolve different representations for different unit types, each evolving in a different sub-population. Second, MOCCEAs explicitly enable more modularity. Rather than searching through a large space, MOECCAs divide the search space of micro for all units, into smaller spaces, one for each unit type and provide the opportunity through shared evaluation, for multiple unit types to evolve interacting co-adapted micro. For example, PPF representation size grows quadratically in the number of unit types without MOCCEA and linearly with MOCCEA.

Our results on the popular Starcraft II research platform, show that we can cooperatively co-evolve micro for a ranged unit using a parameterized control algorithm representation

978-1-7281-2153-6/19/$31.00 ©2019 IEEE

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 23,2026 at 16:43:22 UTC from IEEE Xplore. Restrictions appl

alongside micro for a melee unit using a pure potential fields approach. The two types of units under the control of these cooperatively co-evolved control algorithms work well together to defeat the default Starcraft II AI (SC2AI) playing with 20% more units. We can also use MOCCEA to evolve better micro against a manually co-evolved AI which is significantly better than SC2AI. Furthermore, our multi-objective formulation leads to a pareto front representing multiple tradeoffs between damage done and received and thus more choice to a human or AI player. We believe these results indicate the potential of our method for generating effective micro for multiple types of units in real-time strategy games along our ultimate goal of evolving complete AI for RTS games.

The remainder of this paper is organized as follows. Section II discusses related work in RTS AI research, cooperative co-evolution, and common approaches to evolving micro. Section III describes the representation, and our MOCCEA implementation. In Section IV, we describe our preliminary results and lastly in Section V we draw our conclusions and explain possible future work.

II. Related Work

Significant work has been done over the years in the field of RTS games addressing different aspects of developing an AI player using different techniques [1]. The RTS game community broadly divides RTS game playing into two main categories: macro-management (or simply macro), and micromanagement (or simply micro).There has been increasing interest in research for micro as good micro for skirmishes in a battle has applications in many fields from war-gaming and modeling industrial agents, to robotics, simulations, and video games [4] [5] [6]. Furthermore, good micro can lead to winning skirmishes even when your forces are outnumbered or unfavorably positioned. In this paper, we investigate developing good micro for multiple types of units using MOCCEA.

To develop good micro, researchers have tried many different techniques and we start with methods based on classical machine learning and game tree search. Hladky and Bulitko [7] evaluated hidden semi-Markov models (HSMM) for unit tracking. Sharma et al. [8] used both CBR and reinforcement learning for tactical plan components. Wender and Watson benchmarked different reinforcement learning techniques for micro [9]. Cadena and Garrido [10] used fuzzy CBR (fuzzy case matching) for strategic and tactical planning. Churchill explored a fast search method alpha-beta considering duration (ABCD), a game tree search algorithm for tactical battles in RTS games [11].Aavaas used NEAT to evolve control tactics for group of ranged units in RTS games [12].

Other than machine learning and tree search methods, RTS games researchers have used Influence Maps (IMs) and Potential Fields (PFs) for generating micro. IMs structure a map into grid cells and assigns a value to each grid cell based on enemy unit locations and concentrations and can be used to provide a variety of valuable information. Uriarte and Ontann used IMs for generating kiting in RTS games [13]. Sweetser used an Influence Map (IM) for helping the AI players decision making in their RTS game EmerGEnt [14]. Preuss introduced an IM to optimize group movements in the RTS game Glest [15] [16]. Bergsma and Spronck used IMs for generating adaptive AI in turned-based strategy games. Danielsiek investigated the intelligent moving and path-finding of groups in RTS games using IMs [16]. In this paper, we use IMs to provide the best target location for friendly units.

Potential fields which are usually a distance-dependent vector field generated by a force, were originally used in robotics for smooth group movement [17]. They have also been used for group movement and micro in RTS games. Rathe and Svendsen applied PFs for unit micro in Starcraft [18]. Jonas and Kostler used PFs to control units optimally in StarCraft II for simulating optimal fights [19]. Hagelback and Johansson proposed a multi-agent PF-based bot architecture for the RTS games ORTS and applied PFs for tactical and reactive in the RTS game ORTS [20]. We use PFs in two different ways in this paper for generating group movement and good micro in Starcraft II. Most recently, Louis proposed a method, Pure Potential Fields (PPFs), that produced micro for units maneuvering in 3D within an RTS game to control a single type of unit [3], and Dubey used the same PPF based approach for controlling groups composed of multiple types of units [21]. We use and extend this technique for MOCCEA for controlling multiple types of units.

A co-evolutionary approach for generating an AI agent in games goes back to Shannons work on checkers in the 50s [22] with the most recent notable example being Alphago Zero [23] and AlphaStar [24]. The way species interact with each other during the evolutionary process defines the competitive and cooperative nature of co-evolution - negative interaction for competitive and positive interaction for cooperative. In earlier work, we did competitive coevolution of RTS micro and co-evolved micro from scratch [25]. Iorio used a cooperative co-evolutionary multi-objective algorithm using non-dominated sorting [26]. Antonio presented a cooperative co-evolution method for large-scale multi-objective problems [27]. Both these papers helped establish the strengths and weaknesses of multi-objective, cooperative, co-evolutionary approaches. More relevant to RTS games, Fernndez-Ares used both competitive and cooperative co-evolution for optimization of autonomous agents in RTS games [28].

This paper extends Liu [2] and Louis [3] work in evolving micro for RTS games. Here, we divide the large problem of generating micro for multiple types of units in a group (two unit types in this paper) into two sub-problems: generating micro for each unit type in the group and use two different populations for each sub-problem. One population uses a similar representation to Liu's for ranged units and the other sub-population uses Louis' representation for melee units. Our work is different from both Louis and Liu, in that first, we generate micro for a group composed of multiple types of units rather than for a group composed from a single unit type. Second, rather than using traditional GAs and EMOOA, we use MOCCEA for evolving micro. And third, we believe that being

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 23,2026 at 16:43:22 UTC from IEEE Xplore. Restrictions appl

able to cooperatively co-evolve multiple representations means that we can use the best representation for a particular unit type or for a specific user type. For example, Liu's approach called Meta Search (MS) was designed specifically for hit and run tactics (kiting) for longer ranged units. It hardcodes a parameterized kiting algorithm and evolves these parameters for ranged units. Being more understandable, a hardcoded human designed algorithm suits control applications where safety and liability are important compared with, say a neural network approach, or even compared with Louis' Pure Potential Fields (PPF) approach that evolves parameters for over 13 non-linear potential fields. The next section describes our representations and our MOCCEA implementation.

III. Methodology

The long term goal of our research is to develop a complete human-level RTS game player, and our paper focuses on one aspect of this problem: finding winning micro for multiple types of units in a small skirmish scenario. We build on prior work by Liu [2] and by Louis [3] and extend this work with the purpose of co-evolving micro for multiple types of units (Ranged and Melee unit types) with different representations for each unit type. We thus divide the problem of finding good micro for two types of units by evolving two sub-populations, each searching for good micro for a single unit type. We represent micro for ranged units using a set of parameterized algorithms similar to Liu's Meta Search (MS) [2]. This representation is specifically designed for ranged units, and a good combination of parameter values leads to near-optimal kiting against melee units, good target selection against ranged and melee units, and good retreating (fleeing) against both ranged and melee units. Since MS was not designed for melee units which cannot kite, we represent micro for melee units using a pure potential fields (PPF) approach similar to the approach used by Louis [3]. A good combination of potential field parameters leads to winning micro for melee units. We define both representations below.

A. Meta Search for Ranged Units

Meta search compactly represents micro for ranged units as a combination of one IM, three PFs, and a set of reactive control parameters. Good unit positioning during a skirmish can help friendly units maximize damage dealt to their opponents while keeping relatively safe. An enemy unit influence map gives us such locations. The IM structures a map into 2D grid cells and assigns a value to each grid cell based on enemy unit locations and concentrations. In our work, an IM function computes the influence of an enemy unit at the grid cell occupied by the enemy unit based on a unit weight factor(W_e). The enemy unit influence decreases as a function of distance and becomes zero after R_e distance (in number of grid cells) units. Meta search assumes that the grid cell with the lowest total sum of IM values given by all enemy units in a range R_e and that is closet to enemy unit gives the best attack position. MOCCEAs find the values of W_e and R_e.

Safe navigation to the target location also determines skirmish performance. We use the direction of the vector field generated by attractive and repulsive potential fields to control group movement to locations provided by the IM or by the reactive control parameters. The typical attractive and repulsive potential fields we use come from equation 1 where d is the distance, c_{at} and e_{at} are parameters of the attractive force generated by target position, c_{rf} and e_{rf} are parameters of the repulsive force generated by friendly units, and c_{rw} and e_{rw} are parameters of repulsive force generated by walls. MOCCEAs find good combinations of c_{at}, e_{at}, c_{rf}, e_{rf}, c_{rw} and e_{rw} for smooth, safe unit movements to the target location given by the enemy unit IM.

$$\mathcal{PF} = c_{at}d^{e_{at}} + c_{rf}d^{e_{rf}} + c_{rw}d^{e_{rw}} \quad (1)$$

Variable	W_e	R_e	C_{at}	c_{rf}	c_{rw}	e_{at}	e_{rf}	e_{rw}	S_t	D_k	R_{nt}	D_{bk}	HP_{ef}	HP_{fb}
Bits	5	4	6	6	6	4	4	4	4	5	4	3	3	3
Total	61 bits													

Fig. 1. Meta search chromosome used for ranged units

Once we get close to a target location, selecting a good target and kiting well against melee units increases micro performance. Target selection determines which unit to fire on and is designed to concentrate fire on one target or switch to a new target if there is an enemy candidate with low hit-points nearby. We use R_{nt} and HP_{ef} for determining target selection. Kiting is a technique for speedy ranged units with which the ranged units fire, run out of range, turn back, fire, run out of range, repeatedly dealing maximum damage to enemy units while saving their own health. Three parameters determine kiting behavior and are 1) the distance from a target that a unit starts to kite (D_k), 2) the standstill time before moving after each firing (s_t), and 3) the distance that a unit moves backward during kiting (D_{kb}). Finally, a health-point threshold HP_{fb} determines whether to remove weakened units from the front line to save them for later or not. We then incorporate R_{nt}, HP_{ef}, D_k, s_t, D_{kb}, and HP_{fb} into our algorithms for target selection, fleeing and kiting. The full algorithms can be found in the previous paper by Siming [2]. MOCCEAs tune these parameter values for ranged units and the chromosomal representation for meta search is shown in Figure 1.

B. Pure Potential Fields for Melee Units

Pure potential fields represent micro for melee units using 3 influence maps and 14 potential fields. The IMs' role is to provide safe target locations. Unlike the influence maps used by meta search, the PPF IM function not only considers enemy units' positions in the game world, but also unit health-points and weapon's cool-down for each enemy unit. Within the PPF influence maps, W_1, W_2 and W_2 represent weights for enemy unit positions, enemy unit health-points, and enemy unit weapon cool-down respectively. The IM function computes the influence of an enemy unit at the grid cell occupied by the enemy unit as the weighted linear sum of these factors.

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 23,2026 at 16:43:22 UTC from IEEE Xplore. Restrictions appl

The enemy unit influence decreases as a function of distance by a factor Δf and ceases after R units (again in number of grid cells). Again, the grid cell with the lowest total sum of IM values given by all enemy units and that is closet to the center of enemy units is chosen as the attack location for PPF controlled units. These five parameters (W_1, W_2 , W_2, R, Δf) for the IM function are tuned by MOCCEAs to find good target locations for melee units.

Variable	W_1	W_2	W_3	R	Δf	PF_{arep}	PF_{arfp}	PF_{areh}	PF_{arfh}	PF_{arew}	PF_{arfw}	PF_{at}	PF_{rw}
Total	$33(W_1+ W_2+ W_3+R+\Delta f+ 4PF_{arep}+ 4PF_{arfp} + 4PF_{areh} + 4PF_{arfh} +4 PF_{arew} + 4PF_{arfw} + 2PF_{at} +2PF_{rw})$												

Fig. 2. Pure potential fields chromosome used for melee units

We use potential fields based on the unit positions, unit health, and unit weapon's state to control melee units. We thus define and use attractive and repulsive potential fields generated by enemy unit positions (PF_{arep}), friendly unit positions (PF_{arfp}), enemy unit health (PF_{areh}), friendly unit health (PF_{arfh}), enemy unit weapons (PF_{arew}) and friendly unit positions (PF_{arfw}). The target location, given by the IMs, also exerts an attractive potential field (PF_{at}). Finally, we use a repulsive wall potential (PF_{rw}) to avoid walls and not get trapped in corners. We thus use a total of 14 different potential fields. MOCCEAs tune all the 14 coefficients (within a range of -10000 to 10000) and 14 exponents (within a range of -7 to 8) of the potential fields to control melee units during skirmishes. The 33 unit PPF chromosome (5 IMs parameters + 14 PFs coefficients + 14 PFs exponents) for melee units is shown in the Figure 2.

qWe did experiments to find suitable values for the bounds and the number of steps for both representations that effectively trade-off between complexity and fidelity. Now that we are familiar with the two micro representations, we next describe our MOCCEA implementation.

C. MOCCEA Implementation

The idea of a Cooperative Co-evolutionary Algorithm (CCA) is to separate the components of a problem solution into sub-populations, where each sub-population representing a problem component evolves separately but is evaluated on the component's ability to help cooperatively solve the complete problem [29]. The fitness of an individual solution in each sub-population is measured in terms of cooperation with good solutions from another sub-population in solving the complete problem. We leverage this idea, to solve the problem of finding good micro for two types of units in StarCraft II, a well known and popular RTS game. We separate the components of the problem solution into two sub-populations (say spR for ranged units and spM for melee units), where each sub-population solves the sub-problem of finding good micro for single type of unit. These cooperatively co-evolving sub-populations ultimately give us good micro for multiple types of units.

We denote a solution of the R (for ranged units) sub population, by $RepRU$ and a solution of the sub population M for melee units by $RepMU$. A solution from a sub-population collaborates with a solution from representative solutions of another sub-population to form a candidate solution as shown in Figure 3. We then use our own implementation of Debs well known non-dominated sorting elitist genetic algorithm NSGA-II as our EMOO Algorithm (EMOOA). The integration of CCA with EMOO thus forms MOCCEAs. One of the major challenges we face is how to pick a representative collaborative solution to form a candidate solution to the complete problem. Unlike a single objective CCA, where the representative is simply the current best solution in each sub-population, MOCCEAs can have multiple best solutions (the entire non-dominated pareto front).

We want to keep diversity in our sub-populations while simultaneously keeping evaluation time low. Rather than every member of the pareto front in one sub-population being collaboratively evaluated with every member of the pareto front from another sub-population, we chose one strong solution (that has done maximum damage and received minimum damage) from another sub population for collaborative evaluation. Paying attention to diversity, we then choose two random members of the pareto front from the other sub-population to end up with a fixed total of three representative solutions to be collaboratively evaluated.

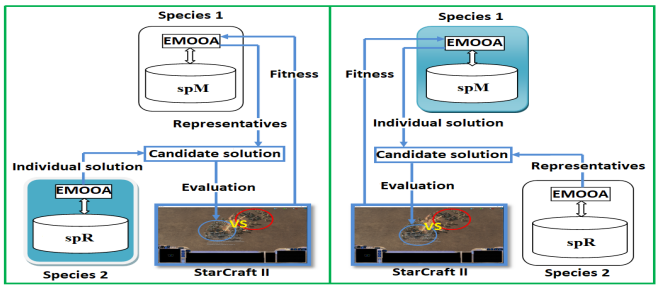


Fig. 3. Typical MOCCEA model from the perspective of each species

D. Fitness Evaluation

During a skirmish we want to maximize damage done to opponent units while minimizing damage taken by friendly units. For this we define a two objective fitness function given by equation 2 that maximizes damage done to enemy units and minimizes damage received by friendly units. This two objective function guides the evolution of each sub-population (spR or spM) with non-dominated selection implemented by NSGAII providing us a pareto front of micro tactics that trade-off between damage done and damage received.

$$Maximize \left[\sum_{enemies} (D_e), \sum_{friends} (1 - D_f) \right] \quad (2)$$

Here, D_e represents damage done to enemy units and D_f represents damage to evolving friendly units. Both the D_e and D_f are normalized between 0 and 1. Note that maximizing (1 - damage received) is the same as minimizing damage

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 23,2026 at 16:43:22 UTC from IEEE Xplore. Restrictions appl

received. This two-objective formulation produces a set of non-dominated solutions (micro) running the range from micro that focuses on fleeing (minimizing damage received) to micro that eliminates the opponents (maximizing damage done) and several trade-offs in between. A human or AI game player now has several choices and can choose the most appropriate tradeoff for a specific game state.

E. Experimental setup

Our experimental setup consists of two major components: the MOCCEA module and the StarCraft II API. We define two sub-populations (spR for ranged units and spM for melee units) inside our MOCCEA module. The MOCCEA is concerned with co-evolving the two sub-populations by running StarCraft II and computing damage done and damage received, then generating new child individuals in their respective sub-populations. The StarCraft II API is responsible for getting candidate solutions (a child individual plus a representative individual from another sub-population) from each sub-population in turn after each generation and evaluating them by playing a game between our Bot against an opponent. The opponent (or bot) is either the default StarCraft II AI or our own Bot. We call our bot, the "baseline" bot and explain how we obtain this bot in section III-E2. The StarCraft II API returns the final fitness of each candidate solution MOCCEA after finishing the game which ends when one of the players has no units or after a set number of frames. As we are interested in micro for two types of units, we set up a skirmish scenario by choosing hellions, a fast ranged Terran unit, as our representative ranged unit and zealots, strong Protoss melee units, as the representative melee unit. Both teams in skirmishes are composes from these two types. All experiments in the paper use hellions for ranged units and zealots for melee units.

1) NSGA II Parameters: We ran a maximum of 500 game frames for all evaluations or until all units on a side were eliminated. Since multiple games are played for each evaluation, we averaged each objective function value to get the two objective function. We used two sub-population of size 20 each and ran the NSGA II for each sub-population for 30 generations. A simple two-point crossover with a probability of crossover of 0.95 and bit mutation with a probability 0.03 worked well for spR which used a binary representation of 61 bits. We used a real-coded encoding with simulated binary crossover (SBX) along with polynomial mutation for spM with 33 parameters. After experimenting with different values, crossover and distribution indexes were both set to 20. We used probabilities of crossover and mutation of 0.90 and 0.05 respectively. Our experimental results and graphs are averaged over 10 runs of the NSGA II with different random seeds.

2) Opponent: Evaluating candidate solutions from respective sub-population requires a good opponent to play against. The opponent should neither be so strong that the candidate solutions can never be beaten nor be so easy that candidate solutions always beat the opponent. In order to produce competitive human micro, we cooperatively co-evolve micro across two different opponents. As the evolution process itself gives us new AIs that can beat the given opponent, instead of hard-coding an opponent; we use a three steps approach (manual competitive co-evolution) to find a better opponent than the SC2AI. We first start with the already existing SC2AI as our opponent and cooperatively co-evolve micro for a group composed of 5 hellions and 25 zealots on our side against 6 hellions and 30 zealots as the opponent. Note that the SC2AI is not very strong, that's why they were given a 20% advantage. Second, we choose one of these final generation individuals (one of the top right points on the graph 9) as our next strong opponent to evolve against. We then use MOCCEA with this micro as our opponent and get a final generation Pareto front. Third, We find the closest individuals to (1.0, 0.5) as our next strong opponent and run MOCCEA against that. After three such repetitions, we designate the resulting individual closest to (1.0, 0.5) as our baseline AI or MCAI for **M**anually **C**oevolved AI. This kind of manual self-play produces a strong opponent, which when evolved against, results in human competitive AI - as shown in the results section. Note that as MCAI has evolved in successive generations by beating the given opponent even when outnumbered by 20%, it does not require 20% advantage as in SC2AI.

3) Training Scenarios: To evaluate one member of the population we created three unit-spawn scenarios determined by initial unit positions. These scenarios are shown in Figures 4, 5, and 6. The first scenario, Scenario1, has opposing sides spawning diagonally across from each other such that our units are positioned near the top right corner of a flat square 120×120 map. In Scenario 2, opposing sides spawn side by side such that our units are positioned near the upper middle half of the same map. Similarly, in Scenario 3, opposing sides spawn diagonally such that enemy units are positioned near the top right corner. In all three scenarios, each side's units are randomly distributed within a radius of 10 map cells. Each candidate solution is evaluated against an opponent in all three scenarios, and each objective's values are averaged over these three scenarios. Although evaluation takes a longer time, the resulting micro performance was better and seemed to generalize well.



Fig. 4. Scenario 1: Diagonal with our units positioned near the top right corner.



Fig. 5. Scenario 2: Side by side with our units positioned near the top middle wall.



Fig. 6. Scenario 3: Diagonal with enemy units positioned near the top right corner.

IV. Results

We cooperatively co-evolved micro for multiple unit types across two different opponents: the default SC2AI and MCAI. Furthermore, we will evaluate robustness of our cooperatively

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 23,2026 at 16:43:22 UTC from IEEE Xplore. Restrictions appl

co-evolved micro across the two different opponents by testing them on three different unseen testing scenarios.

A. Cooperatively Co-evolved Against SC2AI

First, we cooperatively co-evolved micro for hellions and zealots against the existing SC2AI controlling 20% more hellions and zealots in the three training scenarios described earlier (Figures 4, 5, and 6). Specifically, we have 5 hellions and 25 zealots versus SC2AI's 6 hellions and 30 zealots. We have already discussed the population size for both sub-populations and corresponding crossover and mutation parameters earlier in this paper. We ran the MOCCEA where each sub-population takes 30 generations. Figure 7 shows

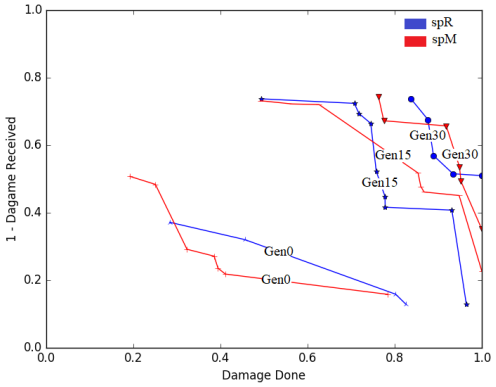


Fig. 7. Ranged (sbR) and melee(spM) units co-cooperatively co-evolving in against SC2AI

the evolution of the pareto fronts of the two cooperatively co-evolving sub-populations (spR and spM) at intervals of fifteen (15) generations for one run of our MOCCEA. Broadly speaking, the pareto front for each sub-population moves towards (1, 0), (1, 1), and (0, 1) on the plot making progress towards maximizing damage done and minimizing damage received. spR evolved micro for ranged units that successfully cooperate with evolving melee units to destroy all opponent units. Unsurprisingly, spM also evolved micro for melee units that successfully cooperated with evolving ranged units and was able to destroy all opponent units. In particular, cooperatively co-evolved micro (let's call this CCAI) was able to beat SC2AI even when outnumbered while saving on average 43% health (49% for spR and 35% for spM) as shown by the solutions on the bottom right of Figure 7.

Figure 8 shows the pareto fronts from 10 runs of the MOCCEA with different random seeds. The figure plots the combined pareto front of both cooperatively co-evolving sub-populations (spR and spM) in the initial generation (Gen0) and the last generation (Gen30) over the ten different random seeds. Note that MOCCEA maintains a diverse set of micro: from fleeing while taking minimal damage (near (0, 1)) to attacking micro that does maximum damage (near (1, 0)). These results provide evidence that we can cooperatively co-

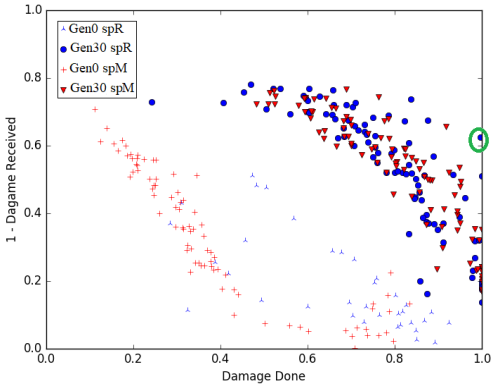


Fig. 8. Initial and final pareto front over 10 runs for spR and spM against SC2AI

evolve a diverse set of micro that learns to perform well against an existing opponent.

B. Cooperatively Co-evolved Against MCAI

We then cooperatively co-evolved micro for groups of 5 hellions and 25 zealots versus an identical MCAI opponent with 5 hellions and 25 zealots in the three training scenarios described earlier (Figures 4, 5, and 6). Note that MCAI is strong enough compared to the default SC2AI that we do not provide it the 20% numbers advantage that we provided the SC2AI. Again, we use the same population size for each sub-population and

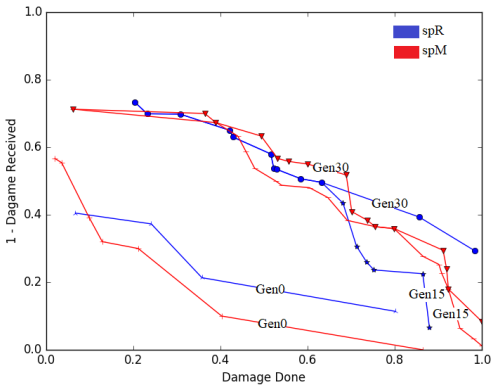


Fig. 9. Ranged (sbR) and melee(spM) units co-cooperatively co-evolving in against MCAI

corresponding crossover and mutation parameters as before. We again run for 30 generations. Figure 9 shows the evolution of the pareto front of the two cooperatively co-evolving sub-populations at intervals of fifteen (15) generations for one run of our MOCCEA module. Again, the spR evolved micro behavior for ranged units that successfully cooperated with evolving melee units and destroyed all the opponent units. Furthermore, spM also evolved micro behavior for melee units that successfully cooperated with evolving ranged units and destroyed all the opponent units.

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 23,2026 at 16:43:22 UTC from IEEE Xplore. Restrictions appl

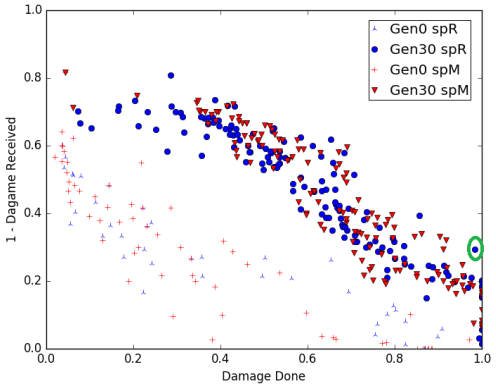


Fig. 10. Initial and final pareto front over 10 runs for spR and spM against MCAI

Again, we ran MOCCEA ten times with different random seeds and Figure 10 shows the result. As before, we got a similar diverse set of micro in the pareto front that trade-offs over damage done and damage received. Similar to the previous CCAI against SC2AI, this CCAI can still eliminate all opponent units, but this time also takes significant damage as shown by the points on the bottom right of Figure 10. This is because our MCAI is significantly stronger than the default SC2AI. In later generation against both SC2AI and MCAI, hellions learned kiting behaviour against zealots and good target selection and fleeing against hellions and zealots. Similarly, zealots learned to herd hellions into corners or into a group that could be encircled. These behaviors can be seen in the videos at https://www.cse.unr.edu/~navin/MOCCEA/.

C. Robustness of CCAI

Next, we consider the robustness of our cooperatively co-evolved micro (CCAI) by testing the CCAI in three scenarios hitherto unseen: random, head-to-head, and surrounded. As we are interested in CCAI doing well against both SC2AI and MCAI, we chose the solutions marked with a green circles in Figures 8 and 10 as our exemplars. We then tested these solutions against SC2AI and MCAI on our three unseen scenarios. These scenarios and our results are described next.

1) Random: In this scenario, opposing sides spawn randomly over the map (Size: 120×120). We tested the CCAI against SC2AI and MCAI on this scenario 50 different times with 50 different initial game unit locations. Results in Table I are averages over these 50 different runs. The first three rows of Table I show that CCAI can eliminate both SC2AI and MCAI in these tests but only while suffering significant damage.

2) Head-to-Head: In this scenario, opposing sides spawn in a line parallel to the y-axis while facing each other. Note that opposing sides are separated by distance greater than their weapon range. Rows 4, 5 and 6 in Table I show the results.

TABLE I
PERFORMANCE OF COOPERATIVELY CO-EVOLVED MICRO (CCAI) AGAINST SC2AI AND MCAI IN DIFFERENT UNSEEN SCENARIOS

CCAI (Co-evolved Against)	Played Against	Scenario	Damage Done	1 - Damage Received
SC2AI	SC2AI	50 Random	0.9188(avg)	0.1088(avg)
MCAI	SC2AI	50 Random	0.9316(avg)	0.1101(avg)
MCAI	MCAI	50 Random	0.9031(avg)	0.1020(avg)
SC2AI	SC2AI	Head-to-Head	1	0.1007
MCAI	SC2AI	Head-to-Head	1	0.3205
MCAI	MCAI	Head-to-Head	1	0.0659
SC2AI	SC2AI	Surrounded	1	0.1185
MCAI	SC2AI	Surrounded	1	0.3346
MCAI	MCAI	Surrounded	1	0.2459

Again, we can see that CCAI can beat both SC2AI and MCAI in these scenarios while suffering damage.

3) Surrounded: In this scenario, opponent units surround our units in a tight circle. Rows 7, 8 and 9 in Table I show the results. Once again, we can see that CCAI can (surprisingly) beat both SC2AI and MCAI even though units are completely surrounded by opponent units. CCAI learns to exploit gaps in encirclement that arise as surrounding units move towards central CCAI units.

CCAI resulting from evolving against MCAI is human competitive against gold level SC2 players in our research lab. Videos of the best attacking CCAI playing against SC2AI and MCAI in the above three scenarios are available at https://www.cse.unr.edu/~navin/MOCCEA/.

V. CONCLUSIONS AND FUTURE WORK

This paper proposed a new approach - multi-objective cooperative co-evolution, for evolving micro for multiple types of units in real-time strategy games. We thus divide the problem of evolving micro for multiple types of units into sub-populations each searching for good micro for a single type of unit. We described a model for evolving such sub-populations as a collection of cooperating species which must be combined in order to form micro for multiple types of units. The cooperation between sub-populations is achieved through the sharing of representatives of sub-populations. Our results indicated that this modular divide and co-adapt approach has significant potential for generating good micro for RTS games.

In this work, we cooperatively co-evolved micro for ranged units using meta search along with micro for melee units using pure potential fields on the popular Starcraft II research platform. Results showed that this cooperatively co-evolved micro for ranged and melee units worked well together to beat default Starcraft II AI, even when outnumbered by 20%. We were also able to generate manually co-evolved AI that is significantly better than SC2AI and beat it to generate human competitive micro for controlling hellions and zealots. Furthermore, we found that our multi-objective fitness function leads to a near-optimal pareto front having different kinds of micro along the front enabling a human or AI player to perform trade-offs that best suits their macro and current state of the game.

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 23,2026 at 16:43:22 UTC from IEEE Xplore. Restrictions appl

We tested the robustness of our cooperatively co-evolved micro on 50 never before seen random scenarios, a randomized head-to-head scenario, and a scenario where opponents surround our units. Results showed that our approach can find micro that performs well on unseen scenarios indicating the generalizability and robustness of the approach.

We believe that our approach thus has potential for generating micro for multiple types of units in a number of ways. First, we can represent and evolve micro based on unit attributes and abilities. Second, due to the modular nature of our model, it should be straightforward to extend this approach to other unit types as well as more unit types in a group. We thus plan to extend this work to more than two unit types as well as different combinations of unit types. We already have good results with co-evolving competetive micro from scratch with MetaSearch [25] and we therefore plan to investigate multi-objective *competitive* cooperative co-evolution to evolve a range of micro that is robust against a range of opposition micro.

VI. Acknowledgments

This work was supported by grant number $N00014-17-1-2558$ from the Office of Naval Research. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Office of Naval Research. Dr. Louis was supported in part by the U.S. Department of Transportation, Office of the Assistant Secretary for Research and Technology (USDOT/OST-R) under Grant No. 69A3551747126 through INSPIRE University Transportation Center (http://inspire-utc.mst.edu) at Missouri University of Science and Technology. The views, opinions, findings and conclusions reflected in this publication are solely those of the authors and do not represent the official policy or position of the USDOT/OST-R, or any State or other entity.

References

[1] S. Ontanón, G. Synnaeve, A. Uriarte, F. Richoux, D. Churchill, and M. Preuss, "A survey of real-time strategy game ai research and competition in starcraft," *IEEE Transactions on Computational Intelligence and AI in games*, vol. 5, no. 4, pp. 293–311, 2013.

[2] S. Liu, S. J. Louis, and C. Ballinger, "Evolving effective micro behaviors in rts game," in *Computational Intelligence and Games (CIG), 2014 IEEE Conference on*. IEEE, 2014, pp. 1–8.

[3] S. J. Louis and S. Liu, "Multi-objective evolution for 3d rts micro," in *Evolutionary Computation (CEC), 2018 IEEE Congress on*. IEEE, 2018, pp. 1–8.

[4] N. R. Jennings, "Commitments and conventions: The foundation of coordination in multi-agent systems," *The knowledge engineering review*, vol. 8, no. 3, pp. 223–250, 1993.

[5] ——, "Controlling cooperative problem solving in industrial multi-agent systems using joint intentions," *Artificial intelligence*, vol. 75, no. 2, pp. 195–240, 1995.

[6] R. Olfati-Saber, J. A. Fax, and R. M. Murray, "Consensus and cooperation in networked multi-agent systems," *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215–233, 2007.

[7] S. Hladky and V. Bulitko, "An evaluation of models for predicting opponent positions in first-person shooter video games." *CIG*, vol. 8, pp. 39–46, 2008.

[8] M. Sharma, M. P. Holmes, J. C. Santamaría, A. Irani, C. L. Isbell Jr, and A. Ram, "Transfer learning in real-time strategy games using hybrid cbr/rl." in *IJCAI*, vol. 7, 2007, pp. 1041–1046.

[9] S. Wender and I. Watson, "Applying reinforcement learning to small scale combat in the real-time strategy game starcraft: Broodwar," in *Computational Intelligence and Games (CIG), 2012 IEEE Conference on*. IEEE, 2012, pp. 402–408.

[10] P. Cadena and L. Garrido, "Fuzzy case-based reasoning for managing strategic and tactical reasoning in starcraft," in *Mexican International Conference on Artificial Intelligence*. Springer, 2011, pp. 113–124.

[11] D. Churchill, A. Saffidine, and M. Buro, "Fast heuristic search for rts game combat scenarios." in *AIIDE*, 2012, pp. 112–117.

[12] A. Gajurel, S. J. Louis, D. J. Méndez, and S. Liu, "Neuroevolution for rts micro," in *2018 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 2018, pp. 1–8.

[13] A. Uriarte and S. Ontanón, "Kiting in rts games using influence maps," in *Eighth Artificial Intelligence and Interactive Digital Entertainment Conference*, 2012.

[14] P. Sweetser and J. Wiles, "Combining influence maps and cellular automata for reactive game agents," in *International Conference on Intelligent Data Engineering and Automated Learning*. Springer, 2005, pp. 524–531.

[15] M. Preuss, N. Beume, H. Danielsiek, T. Hein, B. Naujoks, N. Piatkowski, R. Stuer, A. Thom, and S. Wessing, "Towards intelligent team composition and maneuvering in real-time strategy games," *IEEE Transactions on Computational Intelligence and AI in Games*, no. 2, pp. 82–98, 2010.

[16] H. Danielsiek, R. Stuer, A. Thom, N. Beume, B. Naujoks, and M. Preuss, "Intelligent moving of groups in real-time strategy games," in *Computational Intelligence and Games, 2008. CIG'08. IEEE Symposium On*. IEEE, 2008, pp. 71–78.

[17] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Autonomous robot vehicles*. Springer, 1986, pp. 396–404.

[18] E. A. Rathe and J. B. Svendsen, "Micromanagement in starcraft using potential fields tuned with a multi-objective genetic algorithm," Master's thesis, Institutt for datateknikk og informasjonsvitenskap, 2012.

[19] J. Schmitt and H. Köstler, "A multi-objective genetic algorithm for simulating optimal fights in starcraft ii," in *Computational Intelligence and Games (CIG), 2016 IEEE Conference on*. IEEE, 2016, pp. 1–8.

[20] J. Hagelbäck and S. J. Johansson, "Using multi-agent potential fields in real-time strategy games," in *Proceedings of the 7th international joint conference on Autonomous agents and multiagent systems-Volume 2*. International Foundation for Autonomous Agents and Multiagent Systems, 2008, pp. 631–638.

[21] R. Dubey, J. Ghantous, S. Louis, and S. Liu, "Evolutionary multi-objective optimization of real-time strategy micro," in *2018 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 2018, pp. 1–8.

[22] N. H. Beebe, "A complete bibliography of publications of claude elwood shannon," 2018.

[23] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *nature*, vol. 529, no. 7587, p. 484, 2016.

[24] O. Vinyals, I. Babuschkin, J. Chung, M. Mathieu, M. Jaderberg, W. Czarnecki, A. Dudzik, A. Huang, P. Georgiev, R. Powell *et al.*, "Alphastar: Mastering the real-time strategy game starcraft ii," 2019.

[25] N. K. Adhikari, S. J. Louis, S. Liu, and W. Spurgeon, "Co-evolving real-time strategy game micro," in *2018 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2018, pp. 1990–1997.

[26] A. W. Iorio and X. Li, "A cooperative coevolutionary multiobjective algorithm using non-dominated sorting," in *Genetic and Evolutionary Computation Conference*. Springer, 2004, pp. 537–548.

[27] L. M. Antonio and C. A. C. Coello, "Use of cooperative coevolution for solving large scale multiobjective optimization problems," in *Evolutionary Computation (CEC), 2013 IEEE Congress on*. IEEE, 2013, pp. 2758–2765.

[28] A. Fernández-Ares, A. M. Mora, M. García-Arenas, J. J. M. Guervós, P. García-Sánchez, and P. A. Castillo, "Co-evolutionary optimization of autonomous agents in a real-time strategy game," in *European Conference on the Applications of Evolutionary Computation*. Springer, 2014, pp. 374–385.

[29] M. A. Potter and K. A. D. Jong, "Cooperative coevolution: An architecture for evolving coadapted subcomponents," *Evolutionary computation*, vol. 8, no. 1, pp. 1–29, 2000.

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 23,2026 at 16:43:22 UTC from IEEE Xplore. Restrictions appl